

Memory Efficient Doubly Linked List – Insert At Position –Space Complexity and Algorithm

Program :

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    // Traverse to find the node BEFORE the insertion point
    (pos-1)
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;
    int i = 1;

    while (curr != nullptr && i < pos - 1)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
        i++;
    }

    if (curr == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```

```

    }

    // We are inserting AFTER 'curr'.
    // Let 'next' be the node currently after 'curr'.
    next = XOR(prev, curr->npx);

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Linkage Logic:
    // 1. Setup newNode: npx = curr ^ next
    newNode->npx = XOR(curr, next);

    // 2. Update curr: npx was (prev ^ next), now becomes
    (prev ^ newNode)
    curr->npx = XOR(prev, newNode);

    // 3. Update next (if it exists): npx was (curr ^
    nextNext), now becomes (newNode ^ nextNext)
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted at position " << pos << "."
    << endl;
}

...
case 3:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter position: ";
    cin >> pos;
    insertAtPosition(data, pos);
    break;

```

Space Complexity

1. Input Space Complexity

- **Parameter – only Definition:**
 - **Function Parameters:** *int data and int pos.*
 - **Analysis:** *Both parameters are simple integers. Regardless of the list's size (n) or the target position (pos), these occupy a fixed amount of memory (constant space).*
 - **Complexity:** $O(1)$
- **Full Data Definition:**
 - **Function Data:** *The parameters (data, pos) + the existing XOR linked list.*
 - **Analysis:** *The function operates on a list of n nodes. To reach the insertion point, it may need to traverse up to pos nodes. The entire list is the context for the operation.*
 - **Complexity:** $O(n)$

2. Auxiliary Space Complexity

Definition: Extra memory used by the algorithm during its execution (excluding input).

- **Node Allocation:** *Node *newNode = (Node *)malloc(sizeof(Node));. This allocates memory for exactly **one** new node.*
- **Local Variables:** * Pointers: curr, prev, next, and nextNext.
 - Counter: int i.
 - **Analysis:** *These are local stack variables. Their number does not grow based on the size of the list (n) or the value of pos. They are reused or discarded once the function ends.*
- **Traversal Logic:** *The traversal is iterative (a while loop). It does not use recursion, so it does not add to the call stack as the list size increases.*
- **Bitwise Operations:** *The XOR function performs bitwise math in CPU registers, requiring no extra memory.*

Conclusion: The function uses a constant amount of extra memory to create one node and manage a few pointers, no matter how large the list is.

- **Auxiliary Space Complexity: $O(1)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

Algorithm: Insert at Position (XOR Doubly Linked List)

XORINSPOS(START, ITEM, POS):

*(((Node *)malloc(sizeof(Node)) represents the availability of space
for the data; let's name it AVAIL.))*

1. [VALIDATE POSITION]

If POS < 1, then:

Write: "Invalid position!"

Return and EXIT.

[End of If structure]

2. [INSERT AT BEGINNING?]

If POS = 1, then:

Call XORINSBEG(START, ITEM)

Translating XORINSBEG(START, ITEM) :

XORINSBEG(START, ITEM):

*(((Node *)malloc(sizeof(Node)) represents the availability of
space for the data; let's name it AVAIL.))*

1. [ALLOCATE NEW NODE] Set NEWPTR := AVAIL

2. [MEMORY ALLOCATED?]

If NEWPTR = NULL, then:

Write: "Error: Memory allocation failed."

Return and EXIT.

[End of If structure]

3. [COPY DATA] Set INFO[NEWPTR] := ITEM

4. [INITIALIZE NEW NODE'S POINTER]

Set NEXTXORPTR[NEWPTR] := NULL $\oplus$ START $\left(\begin{array}{l} \text{This sets the new node's} \\ \text{pointer field to the address} \\ \text{of the current head,} \\ \text{as its previous is NULL} \end{array} \right)$

5. [UPDATE OLD HEAD IF LIST NOT EMPTY]

If START $\neq$ NULL, then:

a. Set NEXTTEMPPTR := NULL $\oplus$ NEXTXORPTR[START] $\left(\begin{array}{l} \text{Decode the} \\ \text{address of} \\ \text{the node} \\ \text{following the} \\ \text{current head} \end{array} \right)$

b. Set NEXTXORPTR[START] := NEWPTR $\oplus$ NEXTTEMPPTR $\left(\begin{array}{l} \text{Update the} \\ \text{old head} \\ \text{to point} \\ \text{back} \\ \text{to the new} \\ \text{node} \\ \text{and forward} \\ \text{to its} \\ \text{original} \\ \text{successor} \end{array} \right)$

[End of If structure]

6. [UPDATE HEAD POINTER]

Set START := NEWPTR (Update the global head to the newly created node)

7. [PRINT MESSAGE]

Write: ITEM, " inserted at the beginning. "

8. EXIT

Return and EXIT.

[End of If structure]

3. [INITIALIZE TRAVERSAL]

Set CURRPTR := START

Set PREVPTR := NULL

Set COUNT := 1

4. [TRAVERSE TO POSITION (POS - 1)]

Repeat while CURRPTR $\neq$ NULL AND COUNT $<$ POS - 1:

a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]

b. Set PREVPTR := CURRPTR

c. Set CURRPTR := NEXTTEMPPTR

d. Set COUNT := COUNT + 1

[End of Loop]

5. [CHECK BOUNDS]

If CURRPTR = NULL, then:

Write: "Position out of bounds!"

Return and EXIT.

[End of If structure]

6. [PREPARE NEW NODE AND SURROUNDINGS]

a. Set NEXTTEMPPTR := PREVPTR $\oplus$ NEXTXORPTR[CURRPTR]

b. Set NEWPTR := AVAIL

c. If NEWPTR = NULL, then:

Write: "Error: Memory allocation failed."

Return and EXIT.

d. Set INFO[NEWPTR] := ITEM

7. [LINK NEW NODE]

a. Set NEXTXORPTR[NEWPTR] :

$$= \text{CURRPTR} \oplus \text{NEXTTEMPPTR} \left(\begin{array}{l} \text{New node points back to current} \\ \text{and forward to the} \\ \text{original next node} \end{array} \right)$$

b. Set NEXTXORPTR[CURRPTR] :

$$= \text{PREVPTR} \oplus \text{NEWPTR} \left(\begin{array}{l} \text{Current node's NPX is updated} \\ \text{to point forward to the new node} \end{array} \right)$$

8. [UPDATE NEXT NODE IF IT EXISTS]

If NEXTTEMPPTR $\neq$ NULL, *then*:

a. Set NEXTNEXTTEMPPTR := CURRPTR $\oplus$ NEXTXORPTR [NEXTTEMPPTR]

b. Set NEXTXORPTR [NEXTTEMPPTR] := NEWPTR $\oplus$ NEXTNEXTTEMPPTR

(Next node's NPX is updated to point backward to the new node)

[End of If structure]

9. [PRINT MESSAGE]

Write: ITEM, " inserted at position ", POS

10. EXIT
